

مفهوم العوامل في C++

العوامل (**operators**) عبارة عن رموز لها معنى محدد، و يمكننا تقسيم العوامل إلى 5 مجموعات أساسية كالتالي:

العوامل الرياضية (Arithmetic Operators):

إسم العامل	رمزه	مثال	شرح الكود
Assignment	=	<code>a = b</code>	أعطي <code>a</code> قيمة <code>b</code>
Addition	+	<code>a + b</code>	أضف قيمة <code>b</code> على قيمة <code>a</code>
Subtraction	-	<code>a - b</code>	إطرح قيمة <code>b</code> من قيمة <code>a</code>
Unary plus	+	<code>+a</code>	أضرب قيمة <code>a</code> بالعامل <code>+</code>
Unary minus	-	<code>-a</code>	أضرب قيمة <code>a</code> بالعامل <code>-</code>
Multiplication	*	<code>a * b</code>	أضرب قيمة <code>a</code> بقيمة <code>b</code>
Division	/	<code>a / b</code>	أقسم قيمة <code>a</code> على قيمة <code>b</code>
Modulo	%	<code>a % b</code>	للحصول على آخر رقم يبقى عندما نقسم قيمة <code>a</code> على قيمة <code>b</code>
Increment	++	<code>a++</code>	لإضافة 1 على قيمة <code>a</code> و تستخدم في الحلقات
Decrement	--	<code>a--</code>	لإنقاص 1 من قيمة <code>a</code> و تستخدم في الحلقات

عوامل المقارنة (Comparison Operators):

إسم العامل	رمزه	مثال	شرح الكود
Equal to	==	(a == b)	هل قيمة a تساوي قيمة b ؟ إذا كان الجواب نعم فإنها ترجع true
Not equal to	!=	(a != b)	هل قيمة a لا تساوي قيمة b ؟ إذا كان الجواب نعم فإنها ترجع true
Greater than	>	(a > b)	هل قيمة a أكبر من قيمة b ؟ إذا كان الجواب نعم فإنها ترجع true
Less than	<	(a < b)	هل قيمة a أصغر من قيمة b ؟ إذا كان الجواب نعم فإنها ترجع true
Greater than or Equal to	>=	(a >= b)	هل قيمة a أكبر أو تساوي قيمة b ؟ إذا كان الجواب نعم فإنها ترجع true
Less than or Equal to	<=	(a <= b)	هل قيمة a أصغر أو تساوي قيمة b ؟ إذا كان الجواب نعم فإنها ترجع true

العوامل المنطقية (Logic Operators):

شرح الكود	مثال	رمزه	إسم العامل
هل قيمة <code>a</code> و <code>b</code> تساويان <code>true</code> ؟ هنا يجب أن يتم تحقيق الشرطين ليرجع <code>true</code>	<code>(a && b)</code>	<code>&&</code>	AND
هل قيمة <code>a</code> أو <code>b</code> أو كلاهما تساويان <code>true</code> ؟ هنا يكفي أن يتم تحقيق شرط واحد من الشرطين ليرجع <code>true</code>	<code>(a b)</code>	<code> </code>	OR
هل قيمة <code>a</code> لا تساوي <code>true</code> ؟ إذا كان الجواب نعم فإنها ترجع <code>true</code>	<code>!a</code>	<code>!</code>	NOT

خطوات كتابة برنامج متكامل متكون من عمليات الادخال والاخراج

1-تعريف او التصريح عن المتغيرات التي نحتاجها في البرنامج:

*مفهوم المتغيرات فى C++:

المتغيرات (**variables**) عبارة عن أماكن يتم حجزها في الذاكرة بهدف تخزين بيانات فيها أثناء تشغيل البرنامج.

*أنواع البيانات فى C++:

1- **int** : يستخدم هذا النوع لتعريف عدد صحيح, أي عدد لا يحتوي على فاصلة عشرية مثل:

int x=5;

2- **float** : يستخدم هذا النوع لتعريف عدد يمكن أن يحتوي على فاصلة عشرية .يمكن لهذا العدد أن يحتوي على 7 أرقام بعد الفاصلة مثل:

Float x=10.5;

3- **double** : يستخدم هذا النوع لتعريف عدد يمكن أن يحتوي على فاصلة عشرية .يمكن لهذا العدد أن يحتوي على 15 رقم بعد الفاصلة مثل:

double x=20.6 ;

4- **char** : يستخدم هذا النوع لتعريف حرف اجنبي مثل:

Char x= 'a' ;

5- **string** : يستخدم هذا النوع لتعريف سلسلة رمزية (مجموعة حروف) مثل:

String x="mohammed";

الأحرف المستخدمة في وضع الأسماء في C++:

*أي إسم نضعه لمتغير, دالة, كلاس, كائن إلخ.. يسمى **identifier** في البرمجة.

*اذ يتم التمييز بين العناصر في C++ من خلال أسمائهم, أي من خلال الـ **Identifiers**.

قواعد الزامية عند اعطاء الاسماء:

-البداية بحروف كبيرة مثل (A-Z) او حروف صغيرة (a-z) او الشرطة (_).

-يمنع البدء ب رقم.

- يمنع استخدام القيم true او false.

-يمنع استخدام الكلمات المحجوزة (**Keywords**) الموضحة بالاسفل ↓

- لغة C++ تطبق مفهوم الـ **Case Sensitivity** اي ان الحرف a يختلف عن الحرف A اي ليسوا متغير واحد.

الكلمات المحجوزة في C++

جميع الكلمات التالية محجوزة للغة C++, أي لا يمكن إستخدامها كـ **Identifiers**.

alignas	char16_t	double	if	or	static
alignof and	char32_t	dynamic_cast	inline	or_eq	static_assert
and_eq	class	else	int	private	static_cast
auto	compl	enum	long	protected	struct
bitand	const	explicit	mutable	Public	switch
bitor	constexpr	export	namespace	register	template
bool	const_cast	extern	new	reinterpret_cast	this
break	continue	false	noexcept	requires	thread_local
case	decltype	float	not	return	throw
catch	default	for	not_eq	short	true
char	delete	friend	nullptr	signed	try
	do	goto	operator	sizeof	typedef

typeid
typename
union
unsigned
using
virtual
void
volatile
wchar_t
while
xor
xor_eq

اذن بالنيجة النهائية كل متغير مستخدم بالبرنامج يجب ان يعرف :

```
int a;  
int b;  
float sum;
```

ممكن ان نعرف المتغيرات التي تكون من نفس النوع البياني في سطر واحد:

```
int a,b;  
float sum;
```

2-عملية ادخال او قراءة قيم او بيانات للمتغيرات :

بعد تعريف او التصريح عن المتغيرات نحتاج الى ادخال قيم لهذه المتغيرات وتتم بطريقتين:

-الطريقة الاولى (الادخال المباشر للمتغير) وتتم اثناء (كتابة البرنامج) وتكون بصورتين:

```
int x=5;
```

اي اثناء التصريح في الخطوة الاولى عن المتغير تعطى له القيمة مباشرة.

```
int x;
```

```
x=5;
```

اي بعد التصريح عن المتغير في الخطوة الاولى يتم بعدها كتابة اسم المتغير واسناد له قيمة مباشرة.

-الطريقة الثانية (الادخال غير المباشر للمتغير) وتتم من خلال (شاشة التنفيذ) وتكون بصورتين ايضا:"

```
Cin>>x;
```

```
Cin>>y;
```

هنا الادخال على اكثر من سطر.

```
Cin>>x>>y;
```

هنا الادخال على سطر واحد فقط.

3-عملية المعالجة(اي كتابة المعادلات الرياضية) :

هنا نكتب العملية التي نحتاجها مثلا نحتاج جمع عددين نكتب المعادلة التي من خلالها يتم جمع العددين $x+y$ نحن نعلم هذه المتغيرات المفروض تم تعريفها وادخال قيم لها لذلك نحتاج ل اظهار ناتج هذي المعادلة او العملية الرياضية فنحتاج الى متغير اخر يكون مخزن لناتج جمع x مع y فتكون بالشكل التالي:

$$z=x+y;$$

لنفترض قيمة $x=5$ و $y=5$ اذن قيمة $z=10$

4- الاخراج او الطباعة للقيم :

تطرقنا في المحاضرات السابقة للقواعد الخاصة بعملية الطباعة بالتفصيل.

$$z=x+y;$$

لنفترض انه لدينا

اذن نطبع قيمة المتغير Z والذي يمثل ناتج عملية الجمع بالشكل التالي:

$$\text{Cout}<<z;$$

مثال(1)/برنامج بلغة ++C يستخدم لادخال قيمة متغيرين ويجد ناتج جمعهما ثم يقوم بطباعة ناتج عملية الجمع:

-هنا نلاحظ انه لدينا عملية ادخال قيم وايضا عملية جمع وعملية طباعة ناتج اذن نحتاج الى المضي في الخطوات الاربعة :

1-تعريف او التصريح عن المتغيرات المراد استخدامها في البرنامج.

2-عملية ادخال قيم او قراءة قيم للمتغيرات.

3-المعالجة والتي تمثل كتابة العمليات الرياضية.

4-عملية الاخراج او الطباعة.

```
#include<iostream>
using namespace std;
int main()
{
int x,y,z;
cin>>x;
cin>>y;
z=x+y;
cout<<z;
return 0;
}
```

-ممکن ان يكون الادخال على نفس السطر للبرنامج اعلا وبالشكل التالي:

```
Cin>>x>>y;
```

-يمكن وضع اي عبارة او نص بحيث تكون علامة للشئ المراد قراءته او طباعته :

```
#include<iostream>
using namespace std;
int main()
{
int x,y,z;
cout<<"Enter Value x y:"<<"\n";
cin>>x>>y;
z=x+y;
cout<<"sum="<<z;
return 0;
}
```

مثال(2)/ برنامج بلغة ++C يستخدم لادخال عددين من لوحة المفاتيح ويجاد ناتج قسمة العددين ثم يقوم بطباعة ناتج عملية القسمة:

```
#include<iostream>
using namespace std;
int main()
{
int x,y,z;
cout<<"Enter Value x y:"<<"\n";
cin>>x>>y;
z=x/y;
cout<<"x/y="<<z;
return 0;
}
```

مثال(3)/برنامج بلغة ++C يستخدم لضرب العددين val1=10 , val2=20 ثم يقوم بطباعة الناتج:

```
#include<iostream>
using namespace std;
int main()
{
int val1,val2;
int mul;
val1=10;
val2=20;
mul=val1*val2;
cout<<"mul="<<mul;
return 0;
}
```

مثال(4)/برنامج بلغة ++C يستخدم لضرب وقسمة وجمع وطرح ويجد باقى القسمة بين المتغيرين المستخدم هو الذي يقوم بأدخال قيم المتغيرين:

```
#include<iostream>
using namespace std;
int main()
{
int x,y;
int mul,div,sum,sub,mod;
cout<<"Enter Value : "<<"\n";
cin>>x>>y;
mul=x*y;
div=x/y;
sum=x+y;
```

```

sub=x-y;
mod=x%y;
cout<<"mul="<<mul<<"\n";
cout<<"div="<<div<<"\n";
cout<<"sum="<<sum<<"\n";
cout<<"sub="<<sub<<"\n";
cout<<"mod="<<mod;

return 0;
}

```

مثال (5) / برنامج بلغة ++C يستخدم لحساب محيط ومساحة المستطيل:

```

#include<iostream>
using namespace std;
int main()
{
int length,width;
int area, perimeter;
cout<<"Enter Value : "<<"\n";
cin>>length>>width;
perimeter=2*( length+ width);
area = length * width;
cout<<" perimeter ="<< perimeter<<"\n";
cout<<" area ="<< area;
return 0;
}

```

واجب / برنامج بلغة ++C يستخدم لحساب مساحة المربع ؟؟

مثال 6 / برنامج بلغة ++c يستخدم لحساب مساحة الدائرة ومحيطها ؟؟

علما ان نصف القطر (r) يساوي 5.2 وان النسبة الثابتة $\pi=3.14$

```
#include<iostream>
using namespace std;
int main()
{
float r,pi;
float area, perimeter;
r=5.2;
pi=3.14;
perimeter=2*pi*r;
area = pi*r*r;
cout<<" perimeter ="<< perimeter<<"\n";
cout<<" area ="<< area;
return 0;
}
```

ملاحظة/ في ++c عندما نكتب r^2 نكتبه بالشكل $r*r$

مثال 7 / برنامج بلغة ++C يستخدم لادخال ثلاث درجات لطالب ثم يقوم بأيجاد مجموع الدرجات ومعدل الطالب :

```
#include<iostream>
using namespace std;
int main()
{
int mark1,mark2,mark3;
int sum;
float avg;
cout<<"Enter Value : "<<"\n";
cin>>mark1>>mark2>>mark3;
sum= mark1+mark2+mark3;
avg=sum/3;
cout<<" sum ="<< sum<<"\n";
cout<<" avg ="<< avg;
return 0;
}
```

مثال 8 / برنامج بلغة ++C يقوم بالتبديل بين قيمة متغيرين:

```
#include<iostream>
using namespace std;
int main()
{
int x,y,z;
cout<<"Enter Value : "<<"\n";
cin>>x>>y;
z=x;
x=y;
y=z;
cout<<" x ="<< x<<"\n";
cout<<" y ="<< y;
return 0;
}
```

مثال 9/ برنامج بلغة C++ يقوم بحساب ناتج المعادلة:

$$z = x^2 + \frac{y}{2}$$

//الحل

ملاحظة/ x^2 في برنامج C++ تتفكك الى $x*x$. كذلك الحال اذا كانت x^3 فأنها تتفكك الى $x*x*x$ وهكذا.....

```
#include<iostream>
using namespace std;

int main()
{
    int x,y,z;
    cout<<"Enter Value : "<<"\n";

    cin>>x>>y;
    z=(x*x)+(y/2);
    cout<<" z="<< z;
    return 0;
}
```

واجب 1/ برنامج بلغة C++ يقوم بحساب ناتج المعادلة :

$$z=x+1/y+2$$

واجب 2/ برنامج بلغة C++ يقوم بحساب ناتج المعادلة :

$$e = \frac{2xy}{(x+y)} + \frac{2x}{2(x-z)}$$